

Measured Reliability, Cost and Readiness

What the platform actually does on two corpora, what it costs per task, and what stands between it and production

AgentIA Engineering

2026-09-29

What this document is. The measured state, replacing the estimates in `agentia_first_baseline.tex` (whose central claim has been retracted in-document) and the stale figures in `agentia_current_state.tex`.

Every number here comes from a real run against a real provider with real container verification attempted. Costs are read from the platform's own store (`backend/cost_tracking.db`) and independently confirmed in MLflow. Where a figure could not be measured, this document says so rather than estimating it.

The headline is a null result, and it is the most useful thing here. A task corpus made ten times harder changed nothing: the generator satisfies every encodable rule on its first attempt, every time. That closes the optimization line structurally, and it redirects the effort to the one thing still unmeasured.

Contents

1	What was measured	1
2	Result: the corpus was not the problem	2
2.1	Why the measure cannot vary, at any corpus size	2
3	Result: cost and tokens per task	3
3.1	Hard ten — per task	3
3.2	Per-stage breakdown (one simple service)	3
3.3	Pricing summary	3
3.4	Telemetry: the mirror works, and it agrees	3
4	How this differs from a developer using generative AI	4
4.1	And the honest counterpoints	4
5	Readiness: what stands between this and production	5
6	Recommendation	6

1 What was measured

Two corpora, run end to end on 2026-09-29, provider `deepseek`, model `deepseek-flash`:

Corpus	Size	Character
frozen five	5 blueprints	the held-out set frozen by feature 011; deliberately easy
hard ten	10 blueprints	53 entities, 356 attributes, 80 stories, 207 scenarios; each stresses a different difficulty axis (referential depth, constraint variety, naming collisions, layering temptation, paging, mutation lifecycle, batch scheduling, breadth, invariant-heavy unhappy paths, heterogeneous identifiers)

2 Result: the corpus was not the problem

Measure	frozen five	hard ten
Sessions recorded	5	10
Sessions that generated	5 / 5	10 / 10
Stage runs succeeding on the FIRST request	25 / 25	50 / 50
Correction attempts consumed	0	0
Artifacts per session	15–23	39–63
Conformance score	100 × 5	100 × 10
Conformance density	0.0 × 5	0.0 × 10
Findings in the final artifact sets	none	none

A corpus with ten times the structural complexity produced *identical* conformance. The hypothesis “the tasks are too easy” is falsified.

2.1 Why the measure cannot vary, at any corpus size

This is structural, not statistical, and it is the finding that closes the optimization line.

The compliance gate ****blocks stage-local violations before persistence****. An artifact set that reaches the workspace has therefore already passed every local rule by construction. The only findings that can survive into a saved set are *accumulated* whole-project ones — and exactly one exists (PRINCIPLE_III_CENTRALIZED_ERRORS, the requirement for a `@RestControllerAdvice`), which the generator satisfies by emitting `GlobalExceptionHandler` in the controller stage.

So the final-set conformance measure is pinned at 100 for ****any session that generates at all****. The feature’s own spec anticipated this and could not quantify it:

“because stage-local violations are **blocked before persistence**, the findings that survive into a saved artifact set come from whole-project rules only, which is a **narrow** signal whose real-world variance is unmeasured.”

It is now measured. Variance is **zero**, at both levels of the signal:

Signal level	Observed across all 15 sessions
Final artifact set	100 / density 0.0 / zero findings, in every session
Transient (per-stage introduction)	<i>identical</i> in every session: PRINCIPLE_III_CENTRALIZED_ERRORS × 3, the accumulated whole-project rule firing during the three stages that precede the controller

Consequence for optimization. An objective with no variance has no headroom. No corpus growth and no task difficulty can create it, because the gate removes the local signal before it is stored. Reflective skill optimization is therefore closed *structurally*, not for want of samples.

3 Result: cost and tokens per task

Generation now costs one request per stage — five per session — because nothing fails validation and needs a second attempt. Cost scales with *blueprint complexity* (payload and output tokens), never with retries.

3.1 Hard ten — per task

Task	Calls	Input tok	Output tok	Cost USD
hard-01	5	74,649	107,044	\$0.0745
hard-02	5	55,457	77,352	\$0.0538
hard-03	5	61,780	91,533	\$0.0633
hard-04	5	49,470	75,466	\$0.0518
hard-05	5	49,827	88,788	\$0.0598
hard-06	5	57,412	83,082	\$0.0575
hard-07	5	65,163	98,729	\$0.0681
hard-08	5	78,794	110,118	\$0.0770
hard-09	5	73,428	84,419	\$0.0607
hard-10	5	61,887	101,287	\$0.0691
Total	50	627,867	917,818	\$0.6357

3.2 Per-stage breakdown (one simple service)

Every stage is one request; the test stage is the most expensive because its output is the largest.

Stage	Input tok	Output tok	Cost USD
SCAFFOLDER	2,200	6,620	\$0.0040
DOMAIN	2,669	1,759	\$0.0012
SERVICE	3,250	5,166	\$0.0034
CONTROLLER	3,925	4,210	\$0.0029
TEST	5,711	9,108	\$0.0061
Total	17,755	26,863	\$0.0176

3.3 Pricing summary

Service size	Artifacts	Tokens (in / out)	Cost
simple (minimal)	15	17,755 / 26,863	\$0.0176
medium (frozen five range)	15–23	≈25k / ≈60k	\$0.018–0.025
hard (hard ten range)	39–63	49k–79k / 75k–110k	\$0.052–0.077
per model call (all runs)	—	—	≈ \$0.005

3.4 Telemetry: the mirror works, and it agrees

Cost records are written to the local SQLite store (the system of record) and mirrored per call *and* per session to MLflow. Reading the hard ten back from the tracking server independently reproduced the local totals exactly — 50 runs, \$0.6357 — which is what makes the local figure trustworthy rather than merely self-consistent.

```
# authoritative, offline, deterministic
.venv/bin/python backend/scripts/report_task_costs.py --prefix hard-r1
# the mirror, for a UI
```

```
.venv/bin/mlflow server --backend-store-uri sqlite:///mlflow.db \
  --default-artifact-root ./mlruns --host 127.0.0.1 --port 5000
```

Two defects had to be fixed before these numbers meant anything. Records and cost rows were keyed differently (a fresh session prefix disambiguated the record but not the cost, so a run that spent \$0.1235 reported “cost: no data”), and session aggregates were never mirrored at all — only per-call records. Both are fixed, and the record/cost identity is now pinned by a contract test. Cost telemetry that disagrees with itself is worse than none.

4 How this differs from a developer using generative AI

The honest answer first: **the model is the same commodity model a developer would call.** There is no fine-tuning, no custom weights, and no claim to better code. The difference is not the generator; it is the governed pipeline around it.

	Developer + generic AI	AgentIA
Input	a prompt, refined by conversation	a formal blueprint: entities, attributes, validation rules, BDD acceptance scenarios, frozen before generation
Verification	the developer’s own, by hand, at their discretion	a deterministic rule gate plus a hermetic offline build (<code>mvn test -o</code> , no network) — executable, repeatable, not a judgement call
Failure record	chat history, discarded	a durable per-session record: stage, outcome, requests spent, rule, artifact, severity, and which stage introduced it
Honesty about verification	unknown whether the code was ever run	refuses to substitute a synthetic success ; a session that could not be verified is marked and excluded from every figure
Reproducibility	not reproducible	a deterministic mode that is byte-identical for identical input; model runs carry provider, model, instruction revision and content digests
Cost	a seat licence or an untracked token bill	priced per call, per stage, per session, mirrored, and reportable per task
Self-correction	manual re-prompting, unbounded	bounded at three attempts per stage, recorded, and it stops rather than looping

4.1 And the honest counterpoints

- **The gate never had to fire in these runs.** The model passed every rule first time. The gate’s value is insurance, and insurance looks worthless right up until it does not. Nothing measured here demonstrates that the gate catches errors, because there were none to catch.
- **Conformance is not correctness.** The instrument is a fixed, hand-written rule set. A well-layered but broken service scores full marks.
- **The build and test step is unmeasured.** 0 of 15 sessions were verified: the container runtime was unreachable from the measurement shell. Every session terminated at the sandbox with an

honest “could not verify”. Whether the generated code compiles and passes its tests is therefore *unknown*, and it is the single most important unknown in this document.

- **One product entry point does not use the model.** POST /sessions runs deterministic templates while quick-start and the orchestrator routes run the model (§5). The measurements above describe the model path.

5 Readiness: what stands between this and production

One of the two generation entry points silently does not use the model. The platform has two ways to start a session, and they disagree:

- **Sequential path.** POST /sessions/quick-start with `autoRun: true`, and /orchestrator/pipeline/run, thread the caller’s credentials into `pipeline_runner`, which calls `select_generation_mode(api_key, provider, model_name)` and selects **MODEL**. These routes can use the model, and they are the ones a client should use today.
- **Graph path.** POST /sessions builds its state by calling `select_generation_mode()` *with no arguments* and passes no `llm_api_key`. With no key and no provider that returns **DETERMINISTIC**, so this route emits offline templates and never calls the model — although the graph itself supports MODEL mode, as the batch driver proves by invoking the same graph with credentials in the state.

The consequence is a product surface that behaves differently depending on which endpoint a client picks: one returns model-generated code, the other templates, and nothing in the response distinguishes them. It is a one-line wiring fix, not a blocker — and it is a correction to an earlier draft of this document, which claimed the application could not reach the model at all. That was wrong: the quick-start and orchestrator routes do.

Ranked by what unblocks the most:

Priority	Item	Why it is where it is
P0	Entry-point consistency	POST /sessions silently runs offline templates while quick-start runs the model; a client cannot tell which it received
P1	Build pass rate	the only remaining measurement. Run the corpus where the container works. Until then “reliable” is unfalsifiable
P2	Fix build failures in <i>code</i>	if builds fail, the diagnostic channel names the classes; systematic fixes belong in the harness, the instruction set or the dependency allowlist — deterministic, reviewable, testable — not in prose skills edited by a model
P3	Verification-guided selection (best-of- <i>k</i>)	the evidence-ranked lever. The verifier already exists and is free; generation is \$0.064/task, so $k=4$ is $\approx \$0.26$ and converts unreliable generation into reliable output <i>without trusting the model</i>
P4	Operational hardening	container runtime reachable where it is deployed; per-session cost attribution; the repair cap (the constitution says 3, <code>config.py</code> says 5); the stale README; the two empty skill placeholders — author or delete them

6 Recommendation

1. **Stop the skill-optimization line.** Not for cost — it is \$0.06/session — but because the objective is structurally degenerate (§2.1). Record it as closed on evidence, keep the components on the shelf, and do not grow the corpus for it.
2. **Make the two entry points agree (P0).** Quick-start reaches the model; `POST /sessions` silently does not. A client should not have to know which endpoint yields model output. It is a small wiring fix.
3. **Measure the build pass rate (P1).** One run in a shell where the container runtime initialises. No further reliability work is defensible before it: everything else is optimising a pipeline whose output has never been proven to compile.
4. **If builds fail, fix them in code (P2).** The lesson of this whole exercise is that the reliable place to encode a rule is a deterministic check and a test — three real defects were found this way and none by the language model.
5. **Then, and only then, spend on best-of- k (P3).** It is measurable at this corpus size because it moves a binary per-task outcome, where a conformance nudge of a few points would be buried in noise.
6. **Do not** present conformance as a reliability metric, and do not resurrect the optimizer without first exhibiting a measure that varies.

The one-sentence position

The generator is reliable on the axis that was measured — first-attempt rule-compliance, at \$0.064 per hard service — but the axis that matters for production, whether the output builds and passes its tests, has not been measured once; and one of the application’s two generation entry points never invokes the model at all.